

K-Means – Khi dữ liệu tự tìm về đúng “vị trí” của nó

Trong Machine Learning, có những lúc chúng ta không có nhãn, không có phân loại rõ ràng, thậm chí còn chẳng biết dữ liệu trông ra sao. Nhưng ta vẫn muốn tìm ra một cấu trúc nào đó, một nhóm nào đó tồn tại bên trong. K-Means chính là thuật toán giúp ta làm điều đó — một cách đơn giản mà cực kỳ hiệu quả.

I K-Means là gì?

K-Means là thuật toán *clustering* (phân cụm) trong học không giám sát. Nhiệm vụ của nó rất trực quan: chia dữ liệu thành K cụm, sao cho các điểm trong cùng một cụm giống nhau, và các cụm khác nhau thì cách nhau đủ xa.

Đặc điểm:

- Không cần nhãn trước.
- Mô hình tự tìm pattern.
- Cụm hiện ra rất rõ dù ban đầu bạn không biết điều gì.

II Cách trực quan

II.1 Hoạt động của K-Means

Thuật toán chạy lặp lại hai bước:

1. **Gán cụm:** Các điểm dữ liệu "chọn" centroid gần nhất với chúng.
2. **Cập nhật centroid:** Sau khi biết điểm nào thuộc cụm nào, centroid được tính lại bằng trung bình vị trí của các điểm trong cụm.

Hai bước này lặp đi lặp lại cho đến khi các centroid không còn di chuyển nữa.

II.2 Ví dụ trực quan

Giả sử bạn vào một lớp học mới, chưa biết ai là ai. K-Means sẽ tự động gom:

- Những bạn hay nói chuyện với nhau thành 1 cụm.
- Những bạn thích học chung thành 1 cụm.
- Những bạn trầm tính ngồi một góc thành 1 cụm.

Không ai chỉ dẫn từ trước; mọi thứ tự hình thành dựa trên sự giống nhau.

III Toán học đằng sau

K-Means thực chất là một thuật toán tối ưu hóa, trong đó ta muốn chia dữ liệu thành K cụm sao cho mỗi cụm thật “gọn” và phân tách nhau rõ ràng.

1. Mục tiêu tối ưu

Mục tiêu của K-Means là tối thiểu hóa tổng khoảng cách bình phương Euclidean giữa từng điểm và centroid của nó:

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

Trong đó:

- C_k : tập điểm thuộc cụm k
- μ_k : centroid (tâm cụm)
- $\|x_i - \mu_k\|^2$: bình phương khoảng cách giữa điểm x_i và centroid μ_k

Cụm càng “chặt”, các điểm càng gần centroid $\rightarrow J$ càng nhỏ.

2. Tại sao dùng khoảng cách bình phương?

Dùng bình phương giúp bài toán có nghiệm đẹp:

- Đạo hàm tồn tại.
- Centroid tối ưu chính là trung bình cộng của các điểm trong cụm.

Khi cố định phân cụm C_k , giải đạo hàm theo μ_k :

$$\frac{\partial J}{\partial \mu_k} = 0 \quad \Rightarrow \quad \mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

* Đây là lời giải toán học cho centroid, không phải quy ước.

3. Quá trình tối ưu hai bước lặp

K-Means tối ưu J theo nguyên tắc giảm luân phiên (*coordinate descent*):

1. Assignment Step Giữ nguyên μ_k , mỗi điểm chọn cụm có centroid gần nhất:

$$x_i \in \arg \min_k \|x_i - \mu_k\|^2$$

2. Update Step Giữ nguyên phân cụm, cập nhật lại centroid:

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

Hai bước lặp đi lặp lại cho đến khi cụm không đổi nữa.

Hội tụ

Hai bước trên không bao giờ làm tăng J , và $J \geq 0 \rightarrow$ thuật toán chắc chắn dừng.

Tuy nhiên:

- Dừng tại nghiệm tối ưu cục bộ, không đảm bảo tối ưu toàn cục.
- Nghiệm phụ thuộc vào điểm khởi tạo centroid ban đầu

Vấn đề số cụm K

K-Means không tự tìm số cụm, bạn phải chọn K . Trong thực tế, thường không biết rõ K nên là bao nhiêu.

4. Elbow Method

Elbow Method dựa trên hàm WSS (Within Cluster Sum of Squares):

$$\text{WSS}(K) = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

- Khi K tăng, WSS giảm vì mỗi cụm nhỏ hơn $\rightarrow$ khoảng cách giảm.
- K nhỏ $\rightarrow$ WSS giảm nhanh.
- K lớn $\rightarrow$ WSS giảm chậm dần, vì mô hình gần tối ưu.

Vẽ đồ thị WSS theo K , điểm “gãy khúc” (elbow) là K tối ưu.

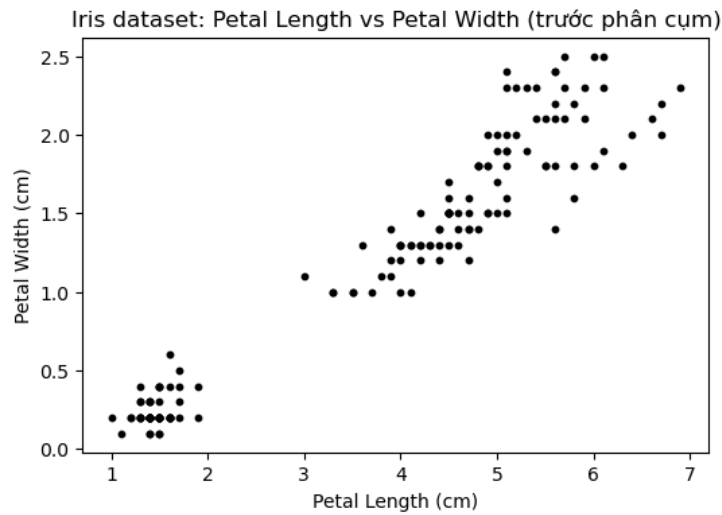
5. Tóm tắt K-Means

K-Means tối ưu một hàm mất mát J bằng cách giảm tổng khoảng cách bình phương từ mỗi điểm đến centroid. Centroid có nghiệm tối ưu là trung bình cộng của các điểm trong cụm, vì thế thuật toán lặp hai bước: gán điểm vào cụm gần nhất và cập nhật centroid. K-Means luôn hội tụ, nhưng nghiệm phụ thuộc điểm khởi tạo và số cụm K . Để chọn K , dùng Elbow Method.

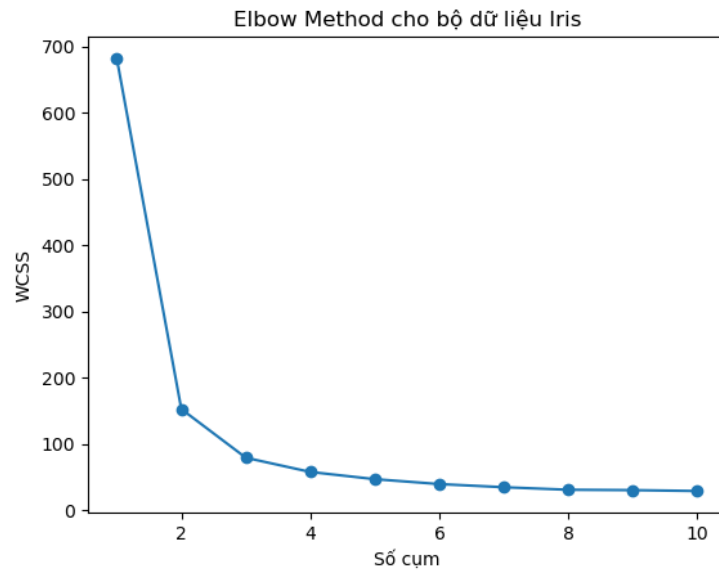
IV Ví dụ trên bộ dữ liệu Iris

Bộ dữ liệu Iris: Gồm 150 mẫu hoa Iris, mỗi mẫu có 4 đặc trưng: chiều dài và chiều rộng của đài hoa, chiều dài và chiều rộng của cánh hoa. Dữ liệu thuộc 3 loài: Iris-setosa, Iris-versicolor và Iris-virginica, mỗi loài 50 mẫu. Bộ dữ liệu cân bằng, có sự khác biệt rõ rệt giữa các loài, thường được sử dụng để minh họa các thuật toán phân loại và phân cụm như K-Means.

1. Hình ảnh về dữ liệu trước khi phân cụm



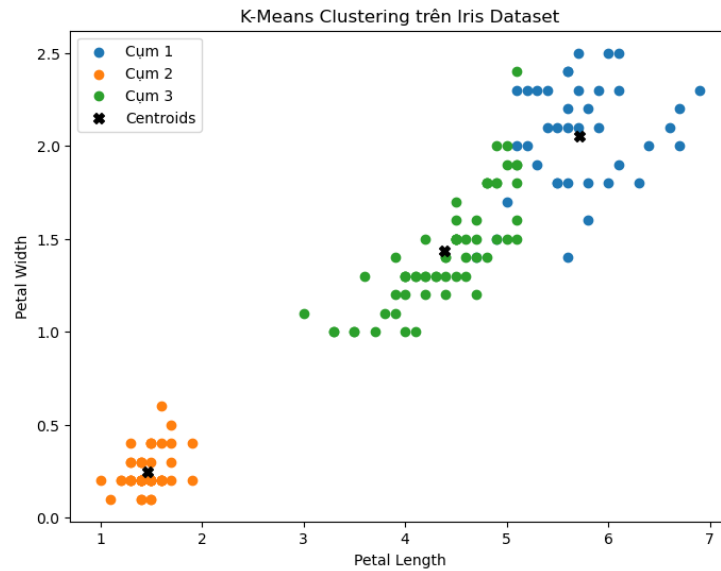
2. Elbow để tìm K cụm cho Iris



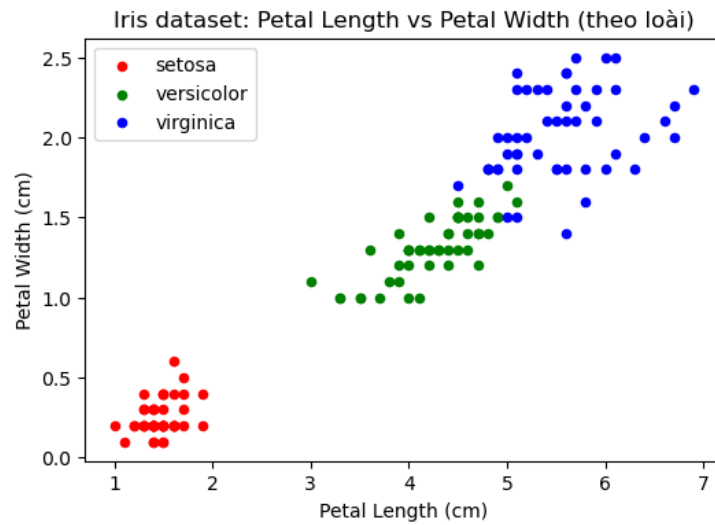
Ta có thể thấy từ 2- >3 WCSS giảm nhiều nhưng từ 3- >4 WCSS giảm rất ít chứng tỏ 3 chính là số cụm cần tìm.

Và trong thực tế thì bộ dữ liệu này có 3 loài hoa nên ở đây việc phân cụm đã chính xác cho 3 loài hoa này

3. Hình ảnh bộ dữ liệu sau phân cụm



4. Hình ảnh thực tế của bộ dữ liệu



V Ưu điểm

- Đơn giản, trực quan: dễ hiểu, dễ triển khai, dễ giải thích
- Nhanh, hiệu quả: chạy tốt với dữ liệu lớn
- Tính toán rõ ràng: centroid = trung bình cộng, dễ minh họa
- Nền tảng cho nhiều thuật toán nâng cao: dùng trong PCA, GMM, các bài toán clustering khác
- Có thể trực quan hóa kết quả: dễ vẽ các cụm trên 2D/3D

VI Nhược điểm

- Phải chọn trước số cụm $K \rightarrow$ nếu chọn sai, kết quả không chính xác
- Nhạy cảm với outlier $\rightarrow$ điểm lạ có thể kéo lệch centroid
- Chỉ tốt với cụm hình dạng tròn/spherical $\rightarrow$ khó xử lý các cụm phức tạp, không đều
- Kết quả phụ thuộc điểm khởi tạo $\rightarrow$ cần chạy nhiều lần hoặc dùng K-Means++

VII Ứng dụng

- Phân nhóm khách hàng (Customer Segmentation) theo hành vi mua sắm, tương tác, thói quen
- Gom ảnh, xử lý hình ảnh (Image Clustering) theo similarity
- Gợi ý bài hát, phim (Recommendation System)
- Phân loại sản phẩm (Retail/Inventory)
- Nghiên cứu sinh học (Bioinformatics): nhóm mẫu, gene, protein
- Tiền xử lý dữ liệu: trước khi áp dụng các thuật toán phức tạp hơn